Chat with Data

In-RAM Single-Server Edition — Architecture, Server Setup & Cost

Technical Architecture Documentation

Deployment Option 1 — everything held in server RAM (stateful)
Scope: Standard and Variance modes
AI Solution Architect: Thiago Alvares
Document Version 1.0 · Prepared July 2026

Contents

1. Overview

Chat with Data is an AI-powered analysis application that answers plain-English questions about uploaded data files. It uses a language model to translate each question into Python (pandas) code, executes that code against the user's data, and then uses the model a second time to turn the computed result into a clear written answer with an optional chart. Every number is computed by executed code, not estimated, and a debug trace makes each step auditable.

This document describes the In-RAM Single-Server Edition — Deployment Option 1. In this edition the Python micro-service keeps all session state (each user's uploaded data, schema, conversation history, and last result) in the server's memory, so follow-up questions are answered quickly without reloading anything from disk or a database. It is the simplest and lowest-cost way to run the application and is well suited to a small user base and a fast go-live. It carries forward the Standard and Variance analysis modes.

This edition is a copy of the base application (the .NET front end + Python micro-service) with only the few files that control single-server, in-memory behavior changed. The analytical core is unchanged.

2. How the Application Works

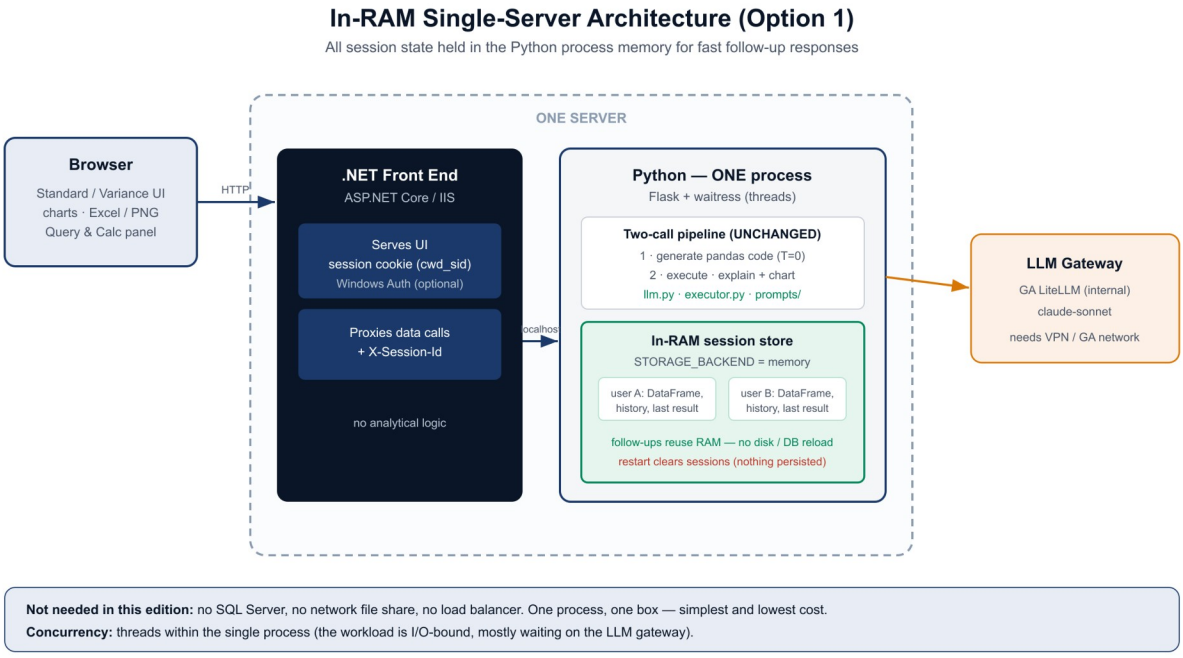

Figure 1 — In-RAM single-server architecture. All session state lives in the Python process memory.

2.1 Two layers

A .NET (ASP.NET Core) front end owns the user interface and the per-user session, and forwards all data work to the Python micro-service over the local network. It contains no analytical logic. The Python micro-service runs the analysis. Keeping the two separate lets each technology do what it does best; in this edition both run on the same server.

2.2 The two-call pipeline

For each question the service makes two model calls. Call 1 (temperature 0) turns the question, the data schema, and recent context into pandas code. That code executes against the in-memory DataFrame(s). Call 2 (temperature 0.3) turns the computed result into a plain-English answer and, when useful, a chart specification. Because the arithmetic is performed by executed pandas code, the numbers are computed rather than estimated.

2.3 In-RAM stateful sessions

The defining characteristic of this edition is that session state is held in the Python process's memory, keyed by the session id the .NET layer supplies in an X-Session-Id header. When a user asks a follow-up question, the service reuses the DataFrame and conversation already in memory — there is no reload from disk or a database — which is what makes responses fast. For this to work the service runs as a single process; concurrency within that process comes from threads, which is efficient here because each request spends most of its time waiting on the LLM gateway.

2.4 Request flow

1. The browser calls the .NET app (upload, ask, export).
2. The .NET app resolves the user's session cookie and forwards the request to the Python service on localhost with the X-Session-Id header.
3. Python runs the two-call pipeline, using and updating the session state held in RAM.
4. The answer, optional chart, and debug trace (or an Excel/PNG file) are relayed back to the browser.

3. Why In-RAM — and the Trade-offs

Holding state in memory gives the fastest possible follow-up experience and the simplest possible infrastructure: no database, no file share, and no load balancer to run. For a small user base this is ideal. The trade-offs are inherent to a single in-memory server and are acceptable at this scale:

- One server, no redundancy — if the server is down, the app is unavailable until it restarts.
- A restart clears active sessions — users re-upload their file. No user data is persisted to disk, which is also a simplicity and privacy benefit.
- It scales up (a larger server), not out. To serve many more users with redundancy, move to the scale-out edition (SQL Server + network file share + multiple instances behind a load balancer).

4. Features

All capabilities are carried over from the validated application and verified in this edition.

Feature	Detail
Standard mode	Plain-English analysis of a single uploaded file.
Variance mode	Compares two labeled datasets on a user-defined key; finds new/missing/changed records and computes variances.
Charts	Automatic Chart.js visuals when useful.

Feature	Detail
Export to Excel	Downloads the last query result as .xlsx.
Export raw query data	Exports the data as the model saw it, multi-sheet when applicable.
Export visuals	Downloads any chart as a PNG image.
Show Query and Calculations	Reveals the generated code, intermediate result, and final answer for full auditability.
Follow-up questions	Reuse the in-RAM data and conversation for fast, contextual answers.

5. Server Configuration Required

Everything runs on one Windows server. The .NET app is hosted by IIS and serves users; the Python service runs as a single-process Windows service and is reachable only on localhost. They communicate over the loopback interface.

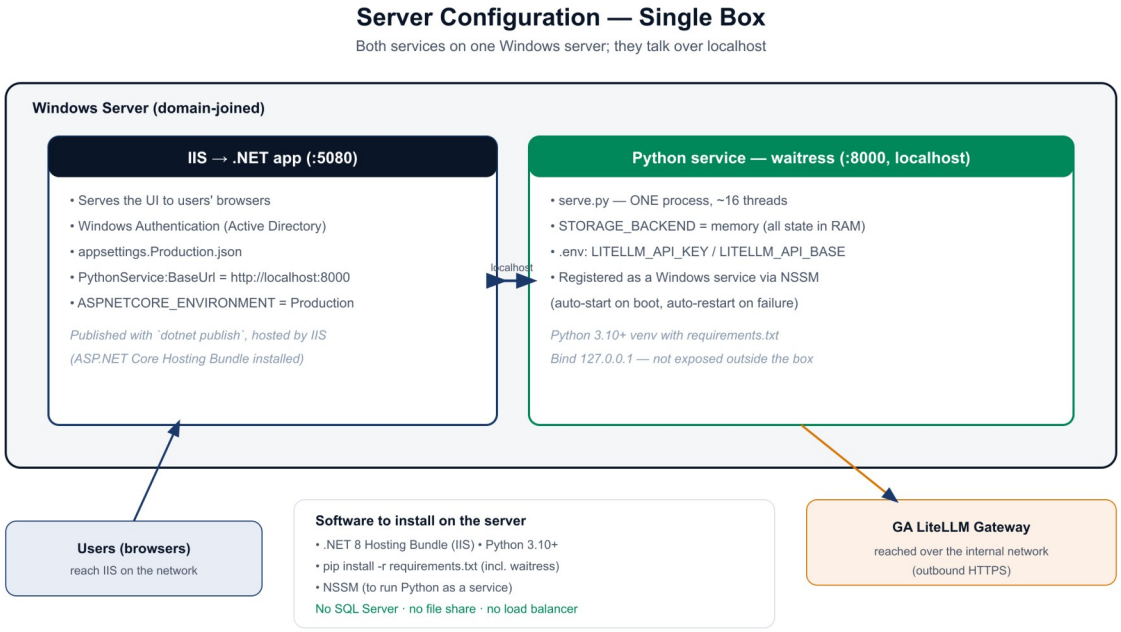

Figure 2 — Single-server configuration. IIS hosts the .NET app; Python runs under waitress as a Windows service on localhost.

5.1 Software to install

- .NET 8 Hosting Bundle (installs the ASP.NET Core module into IIS).
- Python 3.10+ with a virtual environment and the project's requirements.txt (includes waitress).
- NSSM (or an equivalent) to run the Python service as a Windows service.

5.2 Python service

Run the service via serve.py, which uses waitress in a single process with a thread pool (waitress runs on Windows; gunicorn is Linux-only). Bind it to 127.0.0.1:8000 so it is not

exposed outside the box. Register it as a Windows service so it starts on boot and restarts on failure. Set STORAGE_BACKEND=memory (the default) and provide the gateway credentials (LITELLM_API_KEY, LITELLM_API_BASE) in the .env file. Do not run more than one worker/process — that would split the in-memory sessions.

5.3 .NET / IIS

Publish the .NET app and host it in IIS with ASPNETCORE_ENVIRONMENT=Production. Set PythonService:BaseUrl to http://localhost:8000. Enable Windows Authentication (already configured in appsettings.Production.json) once the server is domain-joined, so users are authenticated against Active Directory.

5.4 Networking and security

- Users reach IIS over the internal network; the Python service listens only on localhost.
- Outbound HTTPS from the server to the GA LiteLLM gateway is required for answers.
- Windows Authentication (Active Directory) protects the app; no anonymous access.
- Keep secrets (gateway key) in the server's secret store or a restricted .env, not in source control.

5.5 What is NOT needed

No SQL Server, no network file share, and no load balancer. This is the main reason the edition is simple and inexpensive to run.

6. Sizing

The workload is I/O-bound: each question spends most of its time waiting on the LLM gateway, so a single process with a modest thread pool serves many concurrent users. CPU needs are light. The main sizing driver is memory, because each active user's data sits in RAM. Budget a few hundred megabytes per concurrent active user (depending on file size, up to ~50 MB per file) plus operating-system and IIS headroom. For a small base a server with roughly 4 vCPU and 16-32 GB of RAM is a sensible starting point; increase RAM first if more users are active at once. Because the design scales up rather than out, growth is handled by moving to a larger server (or, past a point, to the scale-out edition).

7. Cost Estimate

This edition is deliberately low-cost: no database, file-share, or load-balancer to license or run. The figures below are approximate, region- and vendor-dependent, and exclude the LLM gateway usage (which runs on the organization's existing gateway and is billed separately by token use, not new infrastructure).

Approach	Indicative cost	Notes
Co-locate on an existing server	$0 incremental	If the current .NET/IIS server has spare CPU and RAM headroom, Python simply runs alongside it. Lowest cost; recommended starting point.
Dedicated VM — small (4 vCPU, 16 GB)	~$120-180 / mo cloud (PAYG); ~$70-110 reserved	On-prem: typically no new spend if virtualization capacity exists. Good for a small base.

Approach	Indicative cost	Notes
Dedicated VM — medium (8 vCPU, 32 GB)	~$250-320 / mo cloud (PAYG)	Headroom for more concurrent active users / larger files.

Software licensing: .NET, Python, waitress, and NSSM are free / open-source. A Windows Server license is typically already owned. There is no SQL Server license in this edition (a cost saving versus the scale-out design).

Recommended starting point: co-locate on the existing .NET server if it has headroom (near-zero added cost); otherwise a single small VM. Scale up the RAM as adoption grows.

8. Operations

- Startup/restart: the Windows service auto-starts the Python process; a restart clears active sessions (users re-upload) — communicate this for planned maintenance.
- Monitoring: watch server RAM (the key metric) and the /health endpoint; the .NET /healthz endpoint reports both layers.
- Backups: none required for application state (nothing is persisted); back up configuration and deployment artifacts as usual.
- Updates: deploy new code, then restart the Python service and recycle the IIS app pool.

9. When to Move Beyond One Server

If usage grows to the point where a single server cannot hold all active users' data in memory, or where redundancy and zero-downtime restarts become requirements, switch to the scale-out edition. That edition keeps the same analytical core but adds SQL Server for session/conversation data, a network file share for uploaded data, and multiple stateless Python instances behind a load balancer. Because the application code is shared, the move is primarily infrastructure and configuration.

10. Summary

The In-RAM Single-Server Edition runs the full Chat with Data experience for the Standard and Variance modes on one server, holding all session state in memory for fast, contextual follow-up answers. It requires no database, file share, or load balancer, which makes it the simplest and least expensive way to go live for a small user base. The trade-offs — a single server and sessions cleared on restart — are appropriate at this scale, and the application can graduate to the scale-out edition later without changing its analytical core.